attribution map (Dombrowski et al., 2019; Ghorbani et al., 2019; Heo et al., 2019; Kindermans et al., 2019; Slack et al., 2020; Zhang et al., 2020)[4]. Perturbation methods present certain issues, such as taking models off their training distribution and eliciting unusual behavior, among other theoretical complications (Feng et al., 2018; Molnar et al., 2021; Hooker et al., 2021; Molnar et al., 2024; Freiesleben & König, 2023; Slack et al., 2021). Developing efficient and accurate attribution methods thus remains an open problem.

**Feature synthesis**. Feature synthesis is a strategy that combines highly activating data set examples and gradient-based attribution methods (Erhan et al., 2009; Szegedy et al., 2014; Olah et al., 2017a). This approach attempts to synthesize inputs that maximize the activation of a component subject to some regularization, such as consistency with a generative model (Nguyen et al., 2016a; 2017) or total variation distance (Mahendran & Vedaldi, 2014). However, criticisms of feature synthesis methods suggest that natural dataset examples may serve interpretation better (Zimmermann et al., 2021; Borowski et al., 2021) or show that current methods struggle to identify trojans (Casper et al., 2023a).

#### 2.1.3b Explanations for the downstream effects of components

Alternatively, the functional role of a component can be described through its downstream effects.

**Studying the direct effect**. The logit lens (Nostalgebraist, 2020) applies the models unembedding matrix to an intermediate residual stream representations, converting it into a distribution over the model's output vocabulary. Direct logit attribution is a generalization of this technique, applying to any appropriately sized vector space in the model, e.g. the output of MLP layers (Geva et al., 2021; 2022b;a; Dar et al., 2023), attention blocks, gradients (Katz et al., 2024) of these, and SDL decoder weights (Bricken, 2023). Adding a trainable affine or linear transformation before unembedding, as also referred to as the tuned lens, improves decoding accuracy (Belrose et al., 2023a; Yom Din et al., 2024), at the cost of less faithfully representing when the model has completed computation.

In the language of causality (Pearl, 2009), unembedding a residual stream vector measures the direct effect of that vector on the output (McGrath et al., 2023). However, the logit lens cannot measure the indirect effect, the effects resulting from the influence that the embedding has on the hidden activations of subsequent layers. Other methods, such as causal interventions (see below), are necessary to measure the indirect effect. In the future, it may be possible to extend logit-lens-like approaches to not only project the effects of network components on directions in output vocabulary space, but also on intermediate downstream components.

**Causal Interventions**. Causal interventions typically substitute ("patch") the value of some network component, usually an activation vector, with a different value during a forward pass, observing the resulting effect on the model. There exist a related set of techniques: ablation (for the special case of zero-ing or otherwise attempting to delete activations entirely), activation patching, causal mediation analysis, causal tracing, and interchange intervention (Vig et al., 2020; Geiger et al., 2020; Meng et al., 2022a; Chan et al., 2022a; Nanda, 2023a). [5]

More surgical patches are also sometimes also made on edges between network components. This approach, known as "path" patching, allows us to isolate the effect of one particular component on only one other component, instead of on the entire rest of the network (Goldowsky-Dill et al., 2023). Causal scrubbing (Chan et al., 2022a) is a generalization of path patching that allows for testing hypotheses concerning any given connection between a set of network components.

Causal intervention methods can also generate supervision signal to identify subspaces of interest. For instance, distributed alignment search (Geiger et al., 2024c) learns a linear subspace that represents a particular concept, using data with interventions on that concept as supervision (Geiger et al., 2024c; Guerner

[4]However, Freiesleben & König (2023) argue that adversarial examples do not truly undermine saliency maps as these are highly dissimilar to real interpretability challenges.

[5]For instance, by patching activations from the corrupt prompt "the capital of Italy is" into the clean prompt "the capital of France is", we can observe the effect on the output ("Rome" vs."Paris"). This tells us which component values are relevant for the differing output between the two prompts, but not the information that remains consistent (e.g. the fact that the answer is a city).

et al., 2024; Wu et al., 2023). Other work learns masks over network components to remove irrelevant components (De Cao et al., 2020; Csordás et al., 2021; Davies et al., 2023).

A causal intervention typically requires a forward pass of the model. This may make performing one for every network component in large models, long contexts, or when using finer-grained components such as sparse autoencoder latents prohibitively expensive. Faster alternatives that work well in practice (Marks et al., 2024; Templeton et al., 2024) include gradient-based approximations to activation patching, such as attribution patching (Nanda, 2023a; Syed et al., 2024), AtP* (Kramár et al., 2024), and integrated gradients (Sundararajan et al., 2017).

**Observing the effects of components on sequential behavior**. Another way to study the effects of model components is to patch in activated components and observe their effect on model behavior. Steering is one example of this (Rimsky et al., 2024; Turner et al., 2024), where components (activation vectors) are activated in order to influence the network's behavior, often in interpretable ways. To determine the functional role of an activation vector, a related method is to have a language model itself decode the activations (Chen et al., 2025; Watkins, 2023; Ghandeharioun et al., 2024; Huang et al., 2024b; Kharlapenko et al., 2024). These approaches, known as "patchscopes", patch activations from one forward pass of a model into a different forward pass (perhaps in a different model). The context of the new forward pass is designed to elicit relevant information from the activations of the original forward pass.

A related sequential behavior based technique is simply to read the chain-of-thought (Wei et al., 2024; Kojima et al., 2024) produced by a language model's output. While this could be considered an 'interpretability technique' in the sense that it aims to explain model decisions, it does not use model internals in those explanations, at least not directly. Recent research demonstrates that chains of thought are not entirely faithful to the model's underlying decision-making process (Agarwal et al., 2024; Atanasova et al., 2023; Turpin et al., 2023; Lanham et al., 2023; Ye & Durrett, 2022). A promising future direction for interpretability research may be to incorporate model internals into chain-of-thought training, which could incentivize faithfulness. Another possibility is building monitors based on model internals to improve transparency in chain-of-thought faithfulness.

#### 2.1.4 Reverse engineering step 3: Validation of descriptions

Initial descriptions of network components' functional roles should be treated as hypotheses that first require validation to ensure that they are reasonable.

Conflating hypotheses with conclusions has regrettably been commonplace in mechanistic interpretability research, making validation an important area for the field to improve (Madsen et al., 2024; Stander et al., 2025). Unfortunately, it is often hard to distinguish faithful explanations of neural network components from merely plausible ones. Numerous examples of model interpretations fail sanity checks (Adebayo et al., 2018; Leavitt & Morcos, 2020; Miller et al., 2024); "interpretability illusions" in which seemingly convincing interpretations of a model later turned out to be false (Bolukbasi et al., 2021; Makelov et al., 2023); or instances where different approaches to explaining the same phenomenon yielded different interpretations (e.g. Chan et al. (2022b) or Chughtai et al. (2023) vs. Stander et al. (2025) vs. Wu et al. (2024a)). Several other instances were previously cited in this review (Section 2.1.3). Hypotheses in interpretability require extensive validation beyond what appearances might imply.

Validating a hypothesis involves posing a simple question: Does the hypothesis make good predictions about the neural network's behavior? Testing the hypothesis often requires multiple approaches (Mueller et al., 2024). To validate descriptions, many approaches simply apply a different description method than the one used to generate the initial hypothesis (Section 2.1.3). If one description method yields a different description from another, it invalidates the hypothesis, which necessitates returning to an earlier step in the reverse engineering cycle (Figure **2**). However, methods for validating descriptions are not limited to other component description methods. Hypothesis validation may take many forms, including the following:

**Predicting activations and counterfactuals**: By using natural language explanations of a given network component's function, it is possible to predict the component's activation levels on different inputs. This analysis can be carried out by humans or by AI systems (Section 2.4), as in (Hernandez et al., 2022; Bills et al.,

2023; Shaham et al., 2025; Juang et al., 2024). In essence, our interpretations should enable us to successfully predict counterfactual scenarios in neural networks. For instance, if we ablate or activate particular network components, we should be able to predict specific downstream effects on other components.

**Predicting and explaining unusual failures or adversarial examples**: Good explanations of neural network behavior should help us identify and explain cases where that behavior fails to produce expected outcomes. For instance, Hilton et al. (2020) validated their methods by explaining cases where a deep reinforcement learning agent's neural network failed to achieve maximum reward and also explained specific hallucinations exhibited by the network. Another approach is to use use an interpretability approach to handcraft an input for a network that functioned as an adversarial example (Carter et al., 2019; Casper et al., 2023c; Mu & Andreas, 2020; Hernandez et al., 2022).

**Handcrafting a network that reconstructs a network behavior**: If our explanations for network behavior are sufficient, we should be able to use them to build replacement parts for the original network. Cammarata et al. (2020) validated their interpretation of a curve detector's function in a convolutional neural network by substituting its parts with simple handcrafted replacements.

**Testing on ground truth**: If the weights of a toy neural network were handcrafted by humans, it is possible to obtain a ground truth explanation for how it works. This proves useful for testing explanations produced by interpretability methods. For example, Conmy et al. (2023) validated a tool's ability to attribute model behaviors to internal components by running it on a simple model that implemented a known algorithm. (See also Paragraph 2.1.4b).

**Using the hypothesis to achieve particular engineering goals**: Another way to test explanations is to assess their utility in downstream applications (Doshi-Velez & Kim, 2017; Casper et al., 2023a). For example, Templeton et al. (2024) discussed examples where manually editing a large language model based on an interpretation led to predictable high-level changes in its behavior. Meanwhile, Marks et al. (2024) showed how an interpretability tool could assist humans with debugging a classifier in a toy task. Farrell et al. (2024) use unlearning (Section 3.2.2) to demonstrate that learned SDL latents don't quite match human concepts, and might not be optimal for particular downstream use cases, highlighting potential issues with SDL.

**Using the hypothesis to achieve specific engineering goals competitively**: Achieving not only useful, but competitive methods sets an even higher standard. For interpretability tools, the highest evaluation criteria require fair comparisons against relevant baselines on real-world tasks instead of cherry-picking them. However, the practice of conducting evaluations using non-cherry-picked tasks remains relatively uncommon. Although attempts have been made to use techniques in the current interpretability toolkit in such evaluations, they have not proven to be consistently useful (Adebayo et al., 2020; Denain & Steinhardt, 2023; Casper et al., 2022b; Hase et al., 2023; Durmus et al., 2024). Some of the most promising research directions are to use interpretability methods to achieve things that would be hard or impossible to achieve without them (Schut et al., 2023). Unless interpretability methods demonstrate that they are competitive with alternative approaches to achieve engineering goals, then the act of demonstrating their usefulness may lead to a bias toward developing methods that only perform well in best-case scenarios and on simple tasks, rather than those that can handle worst-case scenarios and practical challenges.

Interpretability researchers have historically faced challenges in adequately validating their hypotheses due to the high costs in terms of time and cognitive labor. In the following section, we explore two potential solutions that could simplify the validation process: model organisms and interpretability benchmarks.

#### 2.1.4a 'Model organisms' facilitate hypothesis validation.

Although interpretability is sometimes motivated by achieving engineering goals, it is often also approached through the perspective of the natural sciences (Olah et al., 2020b). In certain natural sciences, such as genomics and neuroscience, it is common for researchers to investigate a few extensively studied species known as 'model organisms' or 'model systems'. By conducting in-depth studies on a select group of organisms, like *E. coli*, fruit flies, mice, and macaque monkeys, researchers can leverage the insights and tools gained from those organisms and apply them to other species. For example, imaging specific types of neural activity in